IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

CodeGen-3D: A Benchmark for Evaluating LLMs in Zero-Shot and Iterative 3D Modeling in Blender

HAO JI[1], KOTHA ADITYA [2], SEBASTIAN ESCALANTE [3] and YUNJIAN QIU[4]*,

[1]Center for Advanced Research Computing, University of Southern California, Los Angeles, CA 90007 USA (e-mail: haoji@usc.edu)
[2]Department of Computer Science, University of Southern California, Los Angeles, CA 90007 USA (e-mail: kothaa@usc.edu)
[3]Department of Computer Science, University of Southern California, Los Angeles, CA 90007 USA (e-mail: saescala@usc.edu)
[4]Department of Mechanical Engineering, San Jose State University, San Jose, CA 95112 USA (e-mail: yunjian.qiu@sjsu.edu)

*Corresponding author: YUNJIAN QIU (e-mail: yunjian.qiu@sjsu.edu).

ABSTRACT Large Language Models (LLMs) that generate executable code are opening a new path to text-to-3D by converting natural language prompts into scripts for modeling software like Blender. However, there is a lack of evaluation for such methods currently. Existing benchmarks focus on direct mesh synthesis and overlook critical factors like code execution reliability and human-aligned quality. To address this gap, we introduce CodeGen-3D, a comprehensive benchmark and protocol for assessing 3D modeling via code generation. The benchmark consists of 100 diverse prompts derived from Objaverse with rich captions (Cap3D, DiffuRank, and Llama-based), a standardized Blender execution and six-view rendering pipeline, and a three-part evaluation protocol that assesses: (i) error rate (compilation and runtime failures), (ii) multi-view CLIP alignment (average and max) for semantic similarity, and (iii) a VLM-as-a-Judge pairwise preference task (using GPT-4o) for perceptual quality and prompt adherence. We report results for eight general-purpose LLMs, a specialized BlenderLLM, and an iterative self-correcting agent. Our analysis shows that specialized and iterative systems are substantially more reliable (3–4% failure rate vs. 21–92% for general LLMs) and obtain the highest GPT-4o preference rates (e.g., 33.3% win rate for the iterative agent), even when CLIP scores are similar. This gap between CLIP-based alignment and judge-based preferences underscores the need for feedback-driven, multi-signal evaluation, suggesting that practical text-to-3D systems should combine strong general LLMs with domain specialization or inexpensive self-refinement loops, rather than relying on single-pass code generation. CodeGen-3D establishes a practical, reproducible benchmark and offers strong baselines for future research at the intersection of LLMs, code generation, and 3D modeling.

INDEX TERMS Text-to-3D, Procedural modeling, Code generation, Blender (bpy), Large Language Models (LLMs), Benchmarking and evaluation, CLIP alignment, LLM-as-a-Judge, Iterative agents.

I. INTRODUCTION

The demand for creating high-quality 3D content is rapidly increasing with the rise of immersive experiences in entertainment, XR, and the metaverse [1]. However, 3D asset creation remains a fundamental bottleneck as it is very time-consuming and expensive. Traditional computer-aided design (CAD) workflows, using tools like Blender, depend on laborious manual authoring and thus fails to scale with this demand [2]. Partial solutions like procedural generation, which encodes design rules, are often limited by the need for specialized expertise (e.g., CGA shape grammar), preventing widespread adoption [3].

Recently, Large Language Models (LLMs) have emerged as a natural bridge between language intent and 3D creation due to their strengths in understanding nuanced prompts, planning multi-step procedures, and generating syntactically correct code [4]–[6]. This has sparked a new paradigm of translating text prompts directly into executable scripts that drive 3D software (e.g., Blender via `bpy`), lowering the barrier for procedural modeling for non-experts [7]. Agentic variants further help incorporate self-reflection, iterative correction and enabling advances in tool-use with autonomous skill acquisition in other domains [8]–[10] which can all be used to improve the 3D modeling capabilities of LLMs. However, the evaluation of code-driven text-to-3D generation lags behind these modeling advances. Existing benchmarks

primarily target direct mesh synthesis and rely on geometric or image-space metrics that are ill-suited for procedural code, where many different programs can validly satisfy a given text prompt [11], [12]. As a result, there is no standardized benchmark that jointly measures code execution success, semantic alignment, and human-aligned perceptual quality for LLMs that generate Blender scripts.

Unlike mesh-centric text-to-3D benchmarks such as T^{3}Bench [11] and 3DGen-Bench [12], which evaluate models that directly synthesize 3D geometry using geometric or image-based metrics, CodeGen-3D targets the LLMs that generate executable Blender code. Our benchmark also differs from CADBench [7], which focuses on reconstructing dataset-specific `bpy` scripts and measuring script-level agreement and shape reconstruction accuracy. In contrast, CodeGen-3D is designed for open-ended, prompt-driven generation and standardizes the entire code-to-render pipeline, jointly assessing (i) whether the generated code executes successfully, (ii) how well the resulting renders semantically match the prompt across multiple views, and (iii) how the outputs are perceived by a Vision LLM judge.

Building on this design, we instantiate CodeGen-3D as a benchmark and evaluation protocol for LLMs that generate Blender code from text in an open-ended setting. CodeGen-3D comprises: (i) a curated set of 100 diverse prompts grounded in Objaverse [13] assets and paired with rich captions from Cap3D [14], DiffuRank [15], and LLaMA 3.2 90B Vision [16]; (ii) a standardized Blender execution and six-view rendering pipeline; and (iii) a three-part evaluation consisting of *error rate* (compilation/runtime failures), *multi-view CLIP* (average and max alignment) [17], and a *VLM-as-a-Judge* pairwise preference test for perceptual quality and prompt adherence. Using this setup, we report baselines for eight general-purpose LLMs, a specialized BlenderLLM, and an iterative self-correcting agent.

In particular, this paper makes the following contributions:

- **Benchmark (CodeGen-3D).** We design a *code-first* text-to-3D benchmark with **100** diverse prompts grounded in Objaverse and paired with rich captions (Cap3D, DiffuRank, and Llama-3.2-90B-Vision). A standardized *Blender execution* and *six-view rendering* pipeline ensures apples-to-apples comparisons, and an error taxonomy (syntax/API/runtime) supports precise reliability analysis.
- **Evaluation protocol.** We define a holistic three-part protocol that measures (i) *execution error rate* (compilation/runtime failures), (ii) *multi-view CLIP* alignment (average and max) for semantic similarity, and (iii) a *VLM-as-a-Judge* pairwise preference test (GPT-4o) for perceptual quality and prompt adherence.
- **Baselines and findings.** We report results for **eight** general-purpose LLMs, a specialized *BlenderLLM*, and an *iterative self-correcting agent*. Specialized and iterative systems achieve the lowest failure rates (**3–4**%), whereas general LLMs vary widely (**21–92**%; best: O4-mini **21**%). Despite tightly clustered CLIP scores across models, the judge reveals clear distinctions: the iterative agent secures the most absolute wins overall (**33.3**% win rate; **32** wins), and among general LLMs, GPT-4.1-mini attains the highest win rate (**34.4**%). These results show that reliability and semantic similarity alone understate perceived quality and that *iterative visual feedback* materially improves outcomes.

The remainder of this paper is organized as follows. Section II reviews related work on text-to-3D generation, code-driven CAD pipelines, and evaluation benchmarks. Section III describes the CodeGen-3D benchmark and protocol. Section IV presents the experimental results and analysis. Section V discusses limitations, error modes, and implications. Finally, Section VI concludes and outlines future directions.

II. LITERATURE REVIEW

The creation of three-dimensional digital content has traditionally been the domain of highly skilled artists and engineers using complex software such as Blender, SolidWorks, or AutoCAD. This manual process is notoriously labor-intensive and time-consuming. It also requires substantial training, posing a significant barrier to entry and slowing down iterative design cycles. In recent years, there has been growing interest in using generative artificial intelligence. Advances such as neural radiance fields (NeRFs), the introduction of large-scale 3D datasets, and the maturation of powerful generative models, including diffusion models and LLMs, have catalyzed this revolution. Complementary to our code-first focus, generative models that directly output explicit textured meshes from images have emerged [18], offering high-quality geometry and textures without procedural code. In this section, we review prior work relevant to CodeGen-3D. We first summarize advances in text-to-3D generation via direct synthesis, then discuss programmatic text-to-CAD pipelines powered by LLMs, and finally examine existing evaluation metrics and benchmarks for 3D and CAD generation. Together, these strands highlight the evaluation gap that CodeGen-3D is designed to fill.

A. TEXT-TO-3D VIA DIRECT SYNTHESIS

A parallel line of research has rapidly advanced text-to-3D synthesis by optimizing neural 3D representations under the guidance of 2D diffusion models. The pioneering work DreamFusion introduced score distillation sampling (SDS) to enable zero-shot generation from text prompts [19]. Subsequent methods improved generation quality, speed, and editability through techniques such as coarse-to-fine optimization and more sophisticated distillation strategies (e.g., Magic3D [20], ProlificDreamer [21]). Complementary approaches extend this landscape with direct 3D asset generation (e.g., Point-E [22], Shap-E [23]) and single-image novel-view synthesis (e.g., Zero-1-to-3 [24]). More recently, explicit representations such as 3D Gaussian Splatting (3DGS) have enabled real-time, high-fidelity rendering, opening new possibilities for interactive 3D generation [25]. Benchmarks such

as T3Bench [11] and 3DGen-Bench [12] focus on evaluating these direct text-to-3D models via geometric and multi-view image metrics, but they do not consider code-based generation pipelines or execution reliability.

B. TEXT-TO-CAD GENERATION WITH LARGE LANGUAGE MODELS

A promising paradigm for 3D content creation is programmatic synthesis, which uses the reasoning and code generation capabilities of LLMs, instead of directly synthesizing pixels or mesh data. The models in this category generate executable code, such as Python scripts for Blender or code for parametric CAD software such as OpenSCAD, which is then executed to produce the final 3D model. This approach yields models that are precise, editable, and easily integrated into engineering workflows [7], [26], [27].

Early works explored using general-purpose LLMs like ChatGPT to generate Blender Python (bpy) scripts, demonstrating that strong foundational coding abilities can be highly effective for this task. The field has since evolved toward more autonomous, agentic systems. 3D-GPT [1] utilized a multi-agent framework to interpret instructions and call predefined procedural generators, although its creativity was limited by its fixed function library. A more advanced agent, SceneCraft [28], introduced a dual-loop self-improvement pipeline where a multimodal LLM provides visual feedback to iteratively refine the generated Blender script, enabling the agent to learn a library of "spatial skills".

In the specific domain of Text-to-CAD, several key frameworks have emerged. DeepCAD [29] was an early method that used a transformer-based network to sequentially predict the parameterized command sequences needed to build a CAD model. Building on this, Text2CAD [30] created a custom dataset by generating text descriptions for models from the DeepCAD dataset, which was then used to train a transformer model capable of generating CAD models from text. More recent work has focused on improving LLMs for CAD through self-improvement and novel training strategies. BlenderLLM [7] is a framework that fine-tunes LLMs on bpy scripts and uses a self-improvement loop where the model generates its own training data, which is then filtered for quality and used for further refinement. This work also introduced the BlendNet dataset and the CADBench evaluation suite. Other approaches like CADFusion [31] combine both sequential code signals and visual feedback during training, while CADCodeVerify [26] uses external tools to perform self-correction on the generated code. Most recently, CAD-Llama [32] unifies hierarchical CAD annotations with LLM priors to output professional-grade parametric sequences directly from natural-language instructions, improving complexity and accuracy over general-purpose LLM baselines.

C. EVALUATION OF TEXT-TO-3D AND CAD MODELS

The evaluation of text-to-3D and CAD generation lags behind recent modeling advances, especially for code-driven pipelines, and has become a key bottleneck. Existing approaches can be broadly grouped into three families: image-based metrics, geometry-based metrics, and learned or LLM-based judging, each with important trade-offs.

Image-based metrics render the 3D model from multiple views and apply 2D metrics such as CLIP score, FID, or PSNR [12]. CLIP score is convenient for measuring text–image alignment but has limited knowledge of 3D structure and struggles with spatial and compositional reasoning [33]. FID relies on a 2D Inception network and may not correlate well with human judgments of 3D quality, while PSNR assumes a single ground-truth reference and thus conflicts with the inherently one-to-many nature of generative text-to-3D tasks [34], [35]. Geometry-based metrics such as Chamfer Distance (CD) directly compare 3D geometry but are highly sensitive to outliers, one misplaced vertex can dominate the score and can encourage unnatural "clumping" artifacts that do not reflect human notions of good geometry [34], [36]. Although these classical metrics provide useful quantitative signals, they only partially capture user-perceived quality. Recent work such as WorldCraft [37] highlights the value of leveraging LLM agents for more fine-grained, interactive assessment and tuning of 3D content, narrowing the gap between automated evaluation and user experience.

To address these shortcomings, recent benchmarks increasingly adopt an LLM-as-a-judge paradigm. T^3Bench [11] evaluates text alignment and quality of rendered text-to-3D outputs from multiple views using an LLM-based judge. Gen3DEval [34] employs a Vision Language Model (VLM) fine-tuned to rate text fidelity, surface appearance, and overall quality from multi-view renderings without requiring ground-truth geometry. CADBench [7] is the most relevant benchmark for programmatic synthesis: it measures how well an LLM reproduces ground-truth bpy scripts and shapes using both script-based and image-based validation. However, its emphasis on reconstruction accuracy and dataset-specific scripts differs from our focus on open-ended creative generation and comprehensive evaluation of the full code-to-3D pipeline. In contrast, CodeGen-3D is designed to jointly assess execution reliability, semantic alignment, and human-aligned perceptual quality for LLMs that generate Blender code.

III. METHODOLOGY

To address the above problems, we propose CodeGen-3D, a model-agnostic benchmark that works with any off-the-shelf LLMs. First, each LLM receives a natural-language prompt drawn from our CodeGen-3D dataset, along with instructions for generating the required 3D-modeling code. Next, the generated code is saved as a Python script and executed in Blender to produce .glb file of the model and six orthographic renderings (PNG). Finally, we quantify each model's performance by comparing its rendered outputs against our curated ground-truth models. The remainder of this section elaborates on each stage of the pipeline (i) Ground Truth Data Curation, (ii) Code Generation and Multi-View Rendering, (iii) Evaluation Metrics and (iv) Iterative Refinement Agent. To make

Algorithm 1 CodeGen-3D evaluation pipeline for a given LLM M

Require: Prompt set $\{P_i\}_{i=1}^{N}$, LLM M

1: **for** each prompt P_i **do**
2: Query M with P_i and the Blender system prompt to obtain code s_i.
3: Save s_i as a Python script and execute it in headless Blender.
4: **if** execution fails (syntax/API/runtime error) **then**
5: Record the corresponding failure type and continue to the next prompt.
6: **else**
7: Render six orthographic views $\{I_{i,v}\}_{v=1}^{6}$ and export the 3D model.
8: Compute CLIP similarities between P_i and each view to obtain $S^{\text{avg}}_{\text{CLIP},i}$ and $S^{\text{max}}_{\text{CLIP},i}$.
9: Select the best model view $\hat{I}_i^{\text{model}} = \arg\max_v \text{CLIP}(P_i, I_{i,v})$.
10: Retrieve the best ground-truth view $\hat{I}_i^{\text{gt}}$ (precomputed over ground-truth renders).
11: Query GPT-4o with $(P_i, \hat{I}_i^{\text{gt}}, \hat{I}_i^{\text{model}})$ to obtain a binary preference (ground truth vs. model).
12: **end if**
13: **end for**
14: Aggregate metrics over prompts: error rate, CLIP scores, and VLM preference rate.

the overall procedure explicit, Algorithm 1 summarizes the full CodeGen-3D evaluation pipeline for a given LLM.

Lines 1–4 describe how we query the LLM and execute its code in Blender while logging explicit failure types for our error taxonomy. Lines 6–8 render the six fixed orthographic views and compute multi-view CLIP alignment. Lines 9–10 construct the pairwise comparison inputs to GPT-4o and record the preference used to compute our human-aligned VLM preference rate. The final line aggregates these quantities into the benchmark metrics reported in Section IV.

A. GROUND TRUTH DATA CURATION

To evaluate the LLM-generated models in our benchmark, we first establish a reliable frame of reference by curating a ground-truth dataset of 100 objects sampled from Objaverse 1.0 [13], which comprises roughly 800,000 internet-sourced 3D models. To ensure technical feasibility for the code-generation approaches, we filtered this pool to include only meshes with fewer than 500 faces, avoiding both over-simplified shapes and prohibitively detailed scans (see Figure 1 for examples). We then prioritized categorical diversity by sampling across the 1,156 LVIS-annotated classes in Objaverse, thus assembling a representative set of 100 models without domain-specific bias. Once our ground-truth models were selected, we turned to prompt construction. We began with the original Objaverse model captions from the Cap3D framework [14] and further refined by DiffuRank [15]. Next, to adapt these descriptions to our six-view orthographic renderings, we supplied all six PNG outputs to Llama 3.2 90B Vision and prompted it to synthesize a single comprehensive sentence that captures each object's defining features from every angle. Finally, we selected the best generated caption from the Cap3D and Llama models against its corresponding six-view rendering using CLIP similarity scores and selected the captions with the highest score as the final prompt. We further analyze the impact of caption richness on multi-view CLIP alignment and text–image correspondence in Section V-B, where we treat caption refinement as an ablation on our prompt construction pipeline.

FIGURE 1. Example meshes from Objaverse filtered by face count: *left panel*, fewer than 100 faces; *center panel*, fewer than 500 faces; *right panel*, fewer than 1000 faces.

B. CODE GENERATION AND MULTI-VIEW RENDERING

Given the text prompts generated in the previous stage, we send them to an LLM to produce Blender-compatible Python (bpy) scripts. To ensure consistency and maintain focus on object creation, we provide a standardized system prompt that instructs the LLM to act as an expert Blender scripter and explicitly avoid generating any camera, lighting, or rendering code as shown in Figure 2. Each generated script is then integrated into our rendering template, which defines functions for camera placement and image capture. We then execute the combined script in Blender's headless mode to instantiate the 3D model and produce renderings. To capture every angle, including top and bottom views that a four-view setup would miss, we render six orthogonal images, one along each positive and negative direction of the X, Y, and Z axes.

C. ITERATIVE REFINEMENT AGENT

Beyond our static evaluation metrics, we investigate whether code-driven text-to-3D generation can benefit from self-correction at inference time by designing an iterative refinement agent that utilizes visual feedback to update its code outputs. As illustrated in Figure 3, the agent comprises two collaborating components: a Generator LLM (GPT-4.1) that drafts and revises Blender Python scripts, and a Critic VLM (GPT-4o) that inspects rendered results and issues corrective feedback. The agent itself is implemented as a simple inference-time loop with a fixed number of refinement steps; there is no additional training, reward model, or hand-crafted scoring beyond the signals described below. In Figure 3, numbered labels (1)–(6) indicate the sequential steps of this workflow, from the initial text prompt to the final output.

Each refinement cycle begins with the Generator LLM receiving the original text prompt and producing an initial bpy script. This script is executed in headless Blender to

System Prompt for Blender Code Generation

```
You are an expert in Blender Python scripting.
Generate Python code that creates 3D objects or
scenes in Blender based on the user's prompt.
Focus only on creating the 3D objects - do not
include camera setup, lighting, rendering, or
file saving code.

Your code should:
1. Only create the requested 3D objects using
   Blender's Python API
2. Include necessary imports (bpy, math, random)
3. Use appropriate Blender functions to model,
   texture, and position objects
4. Be clean, efficient, and well-organized

Start with these basic imports:
import bpy
import math
import random

Provide ONLY the raw Python code without markdown
formatting.
```

FIGURE 2. The system prompt used to instruct LLMs to generate 3D models as Blender Python code.

produce a rendered image of the 3D model. The Critic VLM then compares the rendering against the prompt, generating structured, natural-language feedback that pinpoints discrepancies such as incorrect shapes, color mismatches, or missing details (for example, "The glass appears solid, but the prompt specified an empty interior"). The feedback is appended to the original prompt and sent back to the Generator LLM, which incorporates the corrections into a revised script. We repeat this process for up to three iterations, after which the agent returns the final script; in practice, this fixed-budget, fully LLM-driven loop serves as a simple "controller" that alternates between proposing code and critiquing it. Algorithm 2 summarizes this iterative refinement procedure.

Lines 1–4 specify how the generator LLM conditions on the prompt and accumulated feedback to propose a new Blender script at each iteration. Lines 5–7 execute the script and handle failures by turning Blender error logs into textual feedback. Lines 8–13 apply the critic VLM to successful renderings to produce visual feedback that guides the next refinement step. After T iterations, the agent returns the final script without updating any model parameters; all improvements are driven solely by this natural-language feedback loop.

By repeating this process for three iterations, the agent is able to catch and correct errors that single-pass generation often overlooks, ultimately producing models that more faithfully adhere to the user's specification. Importantly, this is a *purely inference-time* procedure, i.e., model parameters are never updated, and the only signal passed across iterations is natural-language feedback. We assess the quality of this feedback indirectly by comparing performance with and without the refinement agent using our benchmark metrics (execution success, CLIP-based alignment, and VLM-as-a-judge preferences; see Section IV); improvements on these metrics, together with qualitative inspection of sampled trajectories, indicate that the feedback is informative and leads to better code. We refer to this pattern as *Self-Refine*, i.e., iterative refinement in which a model (or model pair) alternates between generating an output and generating feedback on that output, then conditions the next attempt on that feedback, following prior work on feedback-driven improvement without training [9].

Algorithm 2 Iterative refinement agent (Self-Refine)

Require: Prompt P, generator LLM G, critic VLM C, max iterations T

1: Initialize feedback $f_0 \leftarrow \emptyset$.
2: **for** $t = 0$ to $T - 1$ **do**
3: Construct generator input from (P, f_t).
4: Use G to generate Blender code s_t.
5: Execute s_t in Blender and render views $\{I_v^{(t)}\}_{v=1}^{6}$.
6: **if** execution fails **then**
7: Derive an error message f_{t+1} from Blender logs and continue.
8: **else**
9: Select a representative view $\hat{I}^{(t)}$ (highest CLIP score with P).
10: **if** $t = T - 1$ **then**
11: **return** final code s_t.
12: **else**
13: Query critic C with $(P, \hat{I}^{(t)})$ to obtain feedback f_{t+1}.
14: **end if**
15: **end if**
16: **end for**

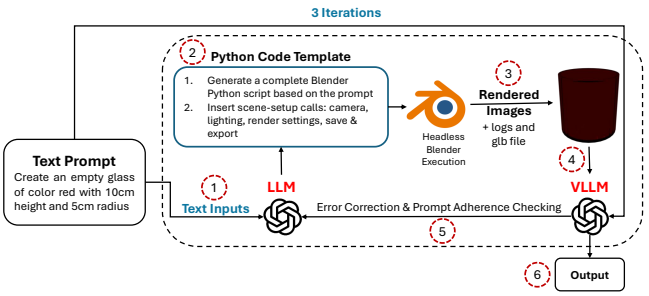

FIGURE 3. Architecture of the iterative refinement agent. (1) A text prompt is provided to the generator LLM, which (2) produces Blender Python code following the given template. (3) Then this code is executed in headless Blender to generate rendered images and logs. (4) The rendered images are passed to a critic VLM together with the prompt, which (5) returns textual feedback for error correction and improved prompt adherence. Steps (2)–(5) are repeated for three iterations, after which (6) the final Blender script and model are returned as the output.

IV. RESULTS

We evaluate the performance of seven distinct llms and our proposed iterative agent on our CodeGen-3D benchmark. Our analysis is structured around three key evaluation pillars: (1)

the raw success rate of code generation, (2) the semantic alignment of the generated models measured by CLIP scores, and (3) the qualitative preference determined by a VLM-as-a-Judge framework.

A. EXPERIMENTAL SETUP

We evaluate eight general-purpose LLMs from OpenAI, Anthropic, and Google via their hosted APIs using the latest generally available snapshots at the time of experimentation, alongside a specialized *BlenderLLM* executed locally and our *Iterative Agent*. All systems receive the same system and task prompts and are instructed to return `bpy`-only code; we exclude camera and lighting code and inject a fixed rendering template to ensure comparability. Where configurable, tool-use/function-calling is disabled. Decoding uses **temperature = 1.0** with all other parameters left at provider defaults (e.g., `top_p`, frequency/presence penalties, sampling strategy). We cap the *maximum generation length* at **4,000 tokens** for non-thinking models and **32,000 tokens** for provider-designated "thinking/reasoning" models. Each model produces one sample per prompt; the Iterative Agent performs three refinement rounds (generator→critic) under the same temperature and per-step budget policy. Blender execution and six-view rendering run on a workstation with a single NVIDIA L40S GPU; caption generation for ground-truth renders uses a small 3×L40S cluster.

Computational Cost. In addition to quality metrics, we report the computational cost of our pipeline. For each model, we measure the wall-clock time required for the *Blender stage* only, i.e., loading and executing the generated Python script inside Blender and rendering all six orthographic views, and we compute the mean and median runtime per successfully generated object. These timings *exclude* the time spent by the LLMs to generate the scripts. For cloud-hosted LLMs, we instead characterize the cost of the generation stage via the API cost per 100 prompts, estimated from provider pricing and the token usage of our experiments. A detailed breakdown of Blender runtimes and API costs is provided in Appendix B.

B. EVALUATION METRICS

To assess the quality and reliability of our LLM-generated models, we employ three complementary metrics: **Error Rate**, **CLIP Score**, and **LLM-as-a-Judge**. These measures evaluate, respectively, code execution success, semantic alignment with the prompt, and human-aligned perceptual preference.

1) Error Rate

First, we define the *error rate* E as the fraction of generated Python (bpy) scripts that fail to compile or execute in headless Blender:

$$E = \frac{N_{\text{fail}}}{N_{\text{total}}},$$

where N_{fail} is the number of scripts that produce syntax errors, API-misuse exceptions, or runtime failures, and N_{total} is the total number of generation attempts. In addition to reporting the overall error rate, we categorize failures into: **Syntax errors:** malformed code or parsing failures, **Semantic/API errors:** incorrect use of Blender's Python API, and **Runtime errors:** invalid object references or execution-time crashes. This breakdown helps identify recurring failure modes and guides future improvements in prompt engineering.

2) CLIP Score

Next, to quantify semantic alignment between the generated model and its text prompt, we compute two CLIP-based metrics over the six rendered views:

$$S_{\text{CLIP}_{\text{avg}}} = \frac{1}{6}\sum_{i=1}^{6} \text{CLIP}(I_i, P), \tag{1}$$

$$S_{\text{CLIP}_{\text{max}}} = \max_{i=1}^{6} \text{CLIP}(I_i, P), \tag{2}$$

where P is the input prompt and $I_1, \ldots, I_6$ are the rendered images. $S_{\text{CLIP}_{\text{avg}}}$ measures overall consistency, while $S_{\text{CLIP}_{\text{max}}}$ ensures that a model is not unduly penalized if key features appear only in one view.

3) LLM-as-a-Judge

Finally, to capture human-aligned judgments of visual quality and prompt adherence beyond what CLIP can measure, we use GPT-4o in a pairwise comparison setup, following recent work showing that GPT-4-class vision–language evaluators are strongly correlated with human preferences and often more human-aligned than CLIP-style metrics in multi-view generative settings [38], [39]. For each object, we select the ground-truth view with the highest CLIP score, and the generated view with the highest CLIP score. These two images are then given to GPT-4o along with the original text prompt. The model is then instructed to choose the image that best matches the description and is more visually appealing as shown in Figure 4. For each prompt i, GPT-4o returns a binary preference label $y_i \in \{\text{gt}, \text{model}\}$ indicating whether the ground-truth or generated image is preferred. We define the *VLM preference rate* of a model M as the fraction of prompts for which the generated image is preferred over the ground truth, i.e.,

$$\text{Pref}(M) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[y_i = \text{model}], \tag{3}$$

where N is the number of prompts and $\mathbb{1}[\cdot]$ is the indicator function. Following the prior work cited above, we treat this VLM preference rate as a proxy for human-aligned perceptual quality and fine-grained prompt compliance.

C. CODE GENERATION SUCCESS RATE

We measure performance by the *error rate*: the percentage of prompts for which a model fails to produce syntactically correct Blender Python that renders a valid 3D object. Figure 5 reports failure rates across ten systems. The fine-tuned **BlenderLLM** [7] and our **Iterative Agent** remain the most reliable, with error rates of **3%** and **4%**, respectively. Among

Prompt Structure for LLM-as-a-Judge

System Prompt: You are an impartial judge. Your task is to evaluate two images based on a description and select the one that better matches the description and is more visually appealing.

User Prompt:
Description: *{object description from Cap3D}*

Please judge which image better follows this description and is more appealing:

- Image 1: Best view of the ground-truth model
- Image 2: Best view of the generated model

FIGURE 4. Pairwise comparison prompt used by GPT-4o to assess perceptual quality and prompt adherence.

general-purpose models, **O4-mini** performs best at **21%**, followed by **GPT-4.1** (**36%**) and **GPT-4.1-mini** (**39%**). The **Gemini 2.5** family exhibits higher error rates, **Pro** at **44%** and **Flash** at **50%**. Anthropic's **Claude Opus** and **Claude Sonnet** are at **54%** and **66%**, and the experimental **GPT 5** has the highest error rate at **92%**. These results suggest that in practical use, one should pair a reasonably strong general LLM with either domain-specific fine-tuning (as in BlenderLLM) or an inexpensive self-refinement loop, rather than relying solely on a single-pass, general-purpose model for generating code for 3D models.

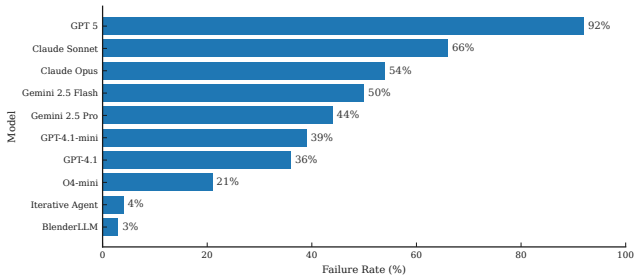

FIGURE 5. Failure rate on the CodeGen-3D benchmark (lower is better). The specialized BlenderLLM [7] and our Iterative Agent achieve the lowest error rates; among general-purpose models, O4-mini is most robust.

D. SEMANTIC ALIGNMENT VIA CLIP SCORE

For all successfully generated objects, we quantify semantic alignment with the input prompt using both *Average* and *Max* CLIP scores (higher is better), as shown in Figure 6. As expected, the **Ground Truth** references yield the highest medians and the tightest interquartile ranges, indicating strong alignment between captions and reference meshes.

Across generative systems, medians are tightly clustered: most models **BlenderLLM** [7], **O4-mini**, **GPT-4.1/GPT-4.1-mini**, **Claude Opus/Claude Sonnet**, and the **Gemini 2.5** variants occupy a narrow band, with substantial overlap in their distributions. BlenderLLM and the stronger OpenAI variants generally sit toward the upper end of this band with relatively compact spreads, suggesting more consistent alignment. Our **Iterative Agent** exhibits a wider dispersion, reflecting higher variance in output quality but also occasional high-scoring samples (visible through longer upper whiskers). The *Max* CLIP distributions mirror the *Average* trend, reinforcing that while several models can produce well-aligned outputs on some prompts, none approach the alignment achieved by the Ground Truth overall.

To assess the robustness of these trends, we conducted paired t–tests and Wilcoxon signed–rank tests comparing each model's CLIP scores to the Ground Truth, as well as Pearson correlations between model and Ground–Truth scores. All models score significantly below Ground Truth ($p < 0.001$ in almost all cases, with GPT 5 at $p < 0.01$ due to its smaller sample size), and correlations are generally weak–to–moderate. Full statistical details, including per–model p–values and correlation coefficients, are provided in Appendix A.

E. QUALITATIVE EVALUATION: VLM AS A JUDGE

To assess qualitative fidelity beyond CLIP, we used **GPT-4o** as a VLM judge for pairwise comparisons. For each successful generation, the judge viewed the best rendering of the generated object and the best view of the ground-truth object, then chose which better adhered to the prompt.

The results in Figure 7 show that the **Ground Truth** is preferred over every model overall. Our **Iterative Agent** secures the most generated wins in absolute terms **32** wins versus **64** ground-truth preferences (**33.3%** win rate). Among general-purpose systems, **GPT-4.1-mini** attains the highest win rate at **34.4%** (21 wins out of 61 comparisons), followed by **Claude Sonnet** (**29.4%**), **Gemini 2.5 Flash** (**28.0%**), **Claude Opus** (**26.1%**), **O4-mini** (**25.3%**), and **GPT-4.1** (**23.4%**). **BlenderLLM** records **17** wins out of 97 (**17.5%**), **Gemini 2.5 Pro** achieves **17.9%** (10/56), and **GPT 5** records no wins. Interestingly, the ranking induced by the VLM judge is not identical to that suggested by CLIP. Some models with competitive CLIP scores lag behind in qualitative wins, while the Iterative Agent outperforms several stronger base models despite similar or slightly lower CLIP medians. This divergence reinforces that explicit reasoning over rendered images and the ability to revise code in responseis an important axis of capability that is not fully captured by static text–image similarity scores. For future systems, this points to two promising directions: incorporating vision–language feedback during training or fine-tuning, and designing inference-time agents that actively query such judges to steer generation toward outputs that better match user preferences.

V. DISCUSSION

A. MODEL OUTPUTS: ERROR MODES AND PERCEPTUAL GAPS

Across identical prompts, LLM-generated outputs often agree on simple *attributes* (e.g., color) but diverge significantly on *structure*, including shape, parts, and their relations. As illustrated in Figure 8, incorporating visual feedback via our Iterative Agent consistently reduces these errors, correcting

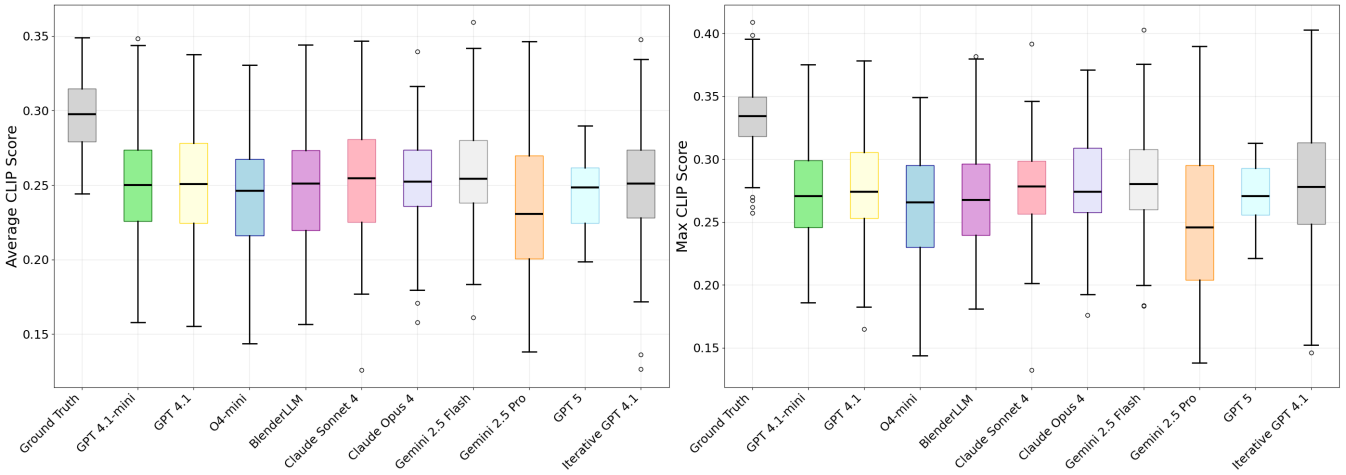

FIGURE 6. **Distributions of *Average* (left) and *Max* (right) CLIP scores for successfully generated objects compared to Ground Truth. Higher values indicate better semantic alignment with the text prompt.**

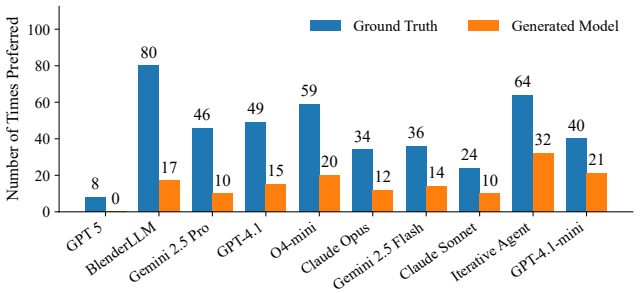

FIGURE 7. **VLM-as-a-Judge pairwise preferences comparing Ground Truth with each model's generations. Bars report the number of times each side was preferred; win rates are noted in text.**

attribute drift and part placement compared to single-pass generation.

Bed. For the *bed* prompt, all methods correctly render a red blanket and white pillows. However, they struggle with the headboard's scale and pillow arrangement. The Iterative Agent leverages feedback to adjust these proportions, resulting in a more plausible assembly.

Dog with bandana. The *dog with bandana* prompt highlights a canonical failure mode where token-level alignment succeeds but structural fidelity fails. Most models generate a black body and red bandana but fail to correctly shape the muzzle or attach the bandana, demonstrating a clear attribute-structure gap.

Red bottle. Similarly, generating a *red bottle* is simple in terms of color, but the systems are distinguished by their ability to model a realistic silhouette, particularly the neck and shoulder geometry.

Red apple. When prompted for a *red apple*, models often default to over-smooth, spherical primitives. While these satisfy the "red" and "smooth" tokens, they typically omit the characteristic stem and subtle asymmetries that define the object's shape.

Takeaways.

- Attribute correctness (e.g., color) is often achieved without structural fidelity (e.g., correct parts).
- Part assembly, including missing or misaligned attachments, is a dominant failure mode.
- Over-smooth geometry can mask structural errors that are noticed by judges but underweighted by CLIP.
- Iterative, vision-in-the-loop refinement improves structure and part placement, even when CLIP score changes are small.

B. RICHER CAPTIONS STRENGTHEN TEXT: IMAGE ALIGNMENT

In this subsection, we analyze how richer, view-aware captions influence CLIP alignment with the ground-truth renders, providing an ablation on the caption refinement procedure described in Section III-A. To test whether richer language anchors semantics to visible evidence, we compared baseline and enhanced captions using a multi-view CLIP improvement test, as illustrated in Figure 9. We define the maximum CLIP score over the six ground-truth views $\{I_v\}$ as:

$$\mathrm{CLIP}_{\max}(c) = \max_{v \in \mathcal{V}} \mathrm{CLIP}\big(c, I_v\big),$$

where an enhanced caption is designated as *improved* if it achieves a higher $\mathrm{CLIP}_{\max}$ score than its baseline.

Earth/globe. For the *earth/globe* model, the enhanced caption specifies visible features like a *transparent surface*, *blue ocean*, and *brown landmasses*. This avoids ambiguous descriptors from the original caption (e.g., "diamond shape") that were not supported by the rendered views.

Wheelbarrow. In the *wheelbarrow* example, adding specific parts (*tray*, *handles*) and colors (*green body*, *red wheel*) increases the grounding between text tokens and corresponding visual regions across views.

Shark. Similarly, naming key geometric features of the *shark*, such as its *pointed snout*, *dorsal and tail fins*, and *gray pattern*, aligns the caption more accurately with the model's geometry and texture.

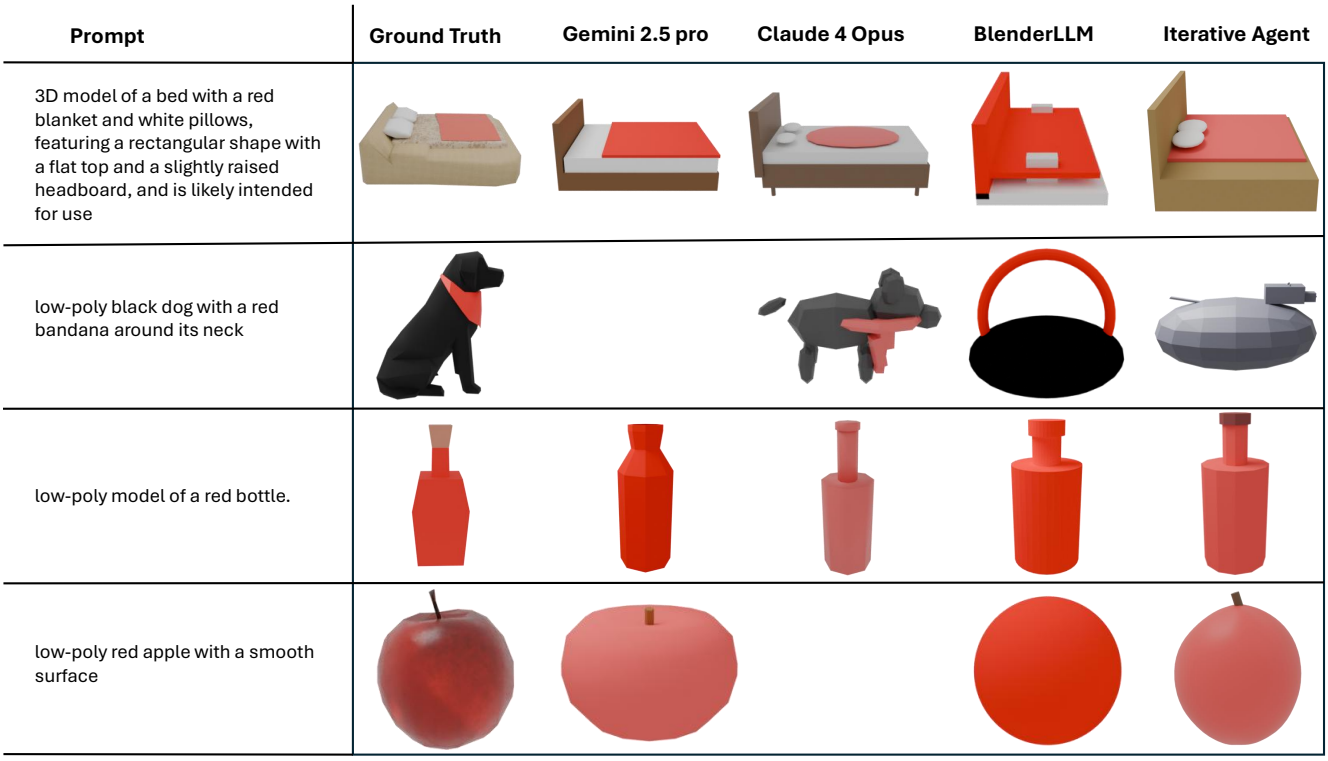

FIGURE 8. Qualitative comparison across systems exposes three recurring error modes. From left to right: prompt, ground truth, Gemini 2.5 Pro, Claude 4 Opus, BlenderLLM, and our Iterative Agent. Rows (top→bottom): bed with red blanket, black dog with red bandana, red bottle, red apple. The most frequent issues are: (i) ***attribute drift*** (color/material correct but shape/proportions off), (ii) ***topology/part errors*** (misplaced/missing parts; incorrect attachments), and (iii) ***over-smooth primitives*** that satisfy color tokens but lose distinctive structure. These patterns mirror the quantitative results: systems can cluster on token-level semantics while diverging in perceived plausibility, which the judge captures more sensitively than CLIP.

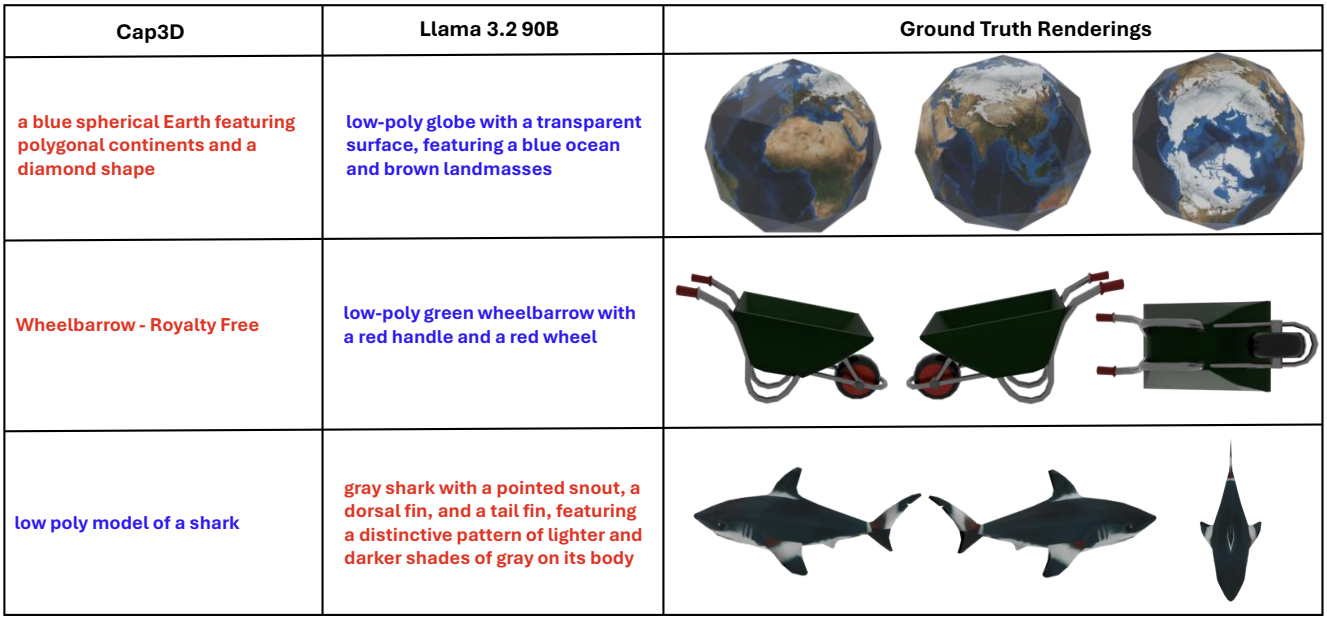

FIGURE 9. Caption upgrades improve view-aware alignment. Left: original Cap3D captions (red). Middle: enhanced captions from Llama 3.2 90B (blue). Right: three ground-truth renderings. A caption is marked **improved** if its multi-view CLIP_{max} against the ground-truth renders increases after enhancement; otherwise it remains **not improved**. Blue captions typically add parts, materials, and spatial relations that match visible evidence (e.g., wheelbarrow tray/handles; shark fins/patterns), reducing ambiguity.

Implications for evaluation.

- Stronger captions make CLIP a more discriminative proxy for evaluating against a reference asset, though it can still overvalue attributes and underweight structure.
- To prevent over-interpretation, CLIP scores should be reported with view aggregation details (e.g., average/max over six views) and paired with reliability and preference metrics.
- For open-ended generation, prompts should include part names, materials, and spatial relations to encourage structural fidelity, not just color or texture matching.

VI. CONCLUSION

In this work, we introduced CodeGen-3D, a benchmark specifically designed to evaluate procedural text-to-3D generation via executable Blender code, filling a gap left by mesh-centric evaluations that overlook execution reliability and perceptual quality. Our framework standardizes the full code→render pipeline and evaluates systems along three complementary axes: execution reliability (error rate with a failure taxonomy), multi-view CLIP semantic similarity (average and max), and human-aligned preference judgments using GPT-4o as a vision–language judge. We curated 100 Objaverse-grounded prompts with rich captions (Cap3D, DiffuRank, Llama 3.2–90B Vision), fixed a six-view rendering protocol, and reported baselines for eight general-purpose LLMs, a specialized BlenderLLM, and an iterative self-correcting agent. Empirically, specialized and iterative systems achieve the lowest failure rates (3–4%), while general LLMs vary widely (21–92%, best 21%); CLIP scores tend to cluster even when perceptual quality differs; and pairwise judge preferences reveal qualitative separations and favor the iterative agent despite mid-pack CLIP. These findings argue for reporting a bundle error rate, CLIP (with view aggregation), and judge preferences, rather than any single metric, and they highlight inference-time feedback as a practical path to improving structural fidelity. We release the benchmark and protocol to support reproducible progress, and foresee extensions to larger and more complex scenes, broader judge pools (including human panels), structure-aware metrics for parts and topology, and additional back ends such as CAD and game engines. For reproducibility, we provide an anonymized GitHub repository with all code, prompts, and evaluation scripts at https://github.com/anonymous/codegen3d, which will be made public upon publication.

A. LIMITATIONS AND FUTURE WORK

While this work establishes a crucial baseline, we acknowledge its limitations. Our benchmark currently consists of 100 prompts, which, while diverse, could be expanded to cover an even wider range of object categories and complexities. Furthermore, our qualitative evaluation relies on a single VLM (GPT-4o) as a judge, which may have its own intrinsic biases; future work could incorporate a panel of different VLMs and human evaluators to further validate these findings.

Based on our results, we propose several directions for future research. Expanding the CodeGen-3D benchmark with more complex prompts involving multi-object scenes and complex material properties is a clear next step. There is also a significant opportunity to develop more sophisticated agentic architectures. Our results suggest that iterative, feedback-driven refinement is a highly promising path toward generating not just semantically similar, but qualitatively excellent 3D models. Finally, this evaluation framework could be extended to other procedural generation domains beyond Blender, such as CAD software or game development engines, providing a universal tool for assessing the future of AI-driven design.

ETHICS AND RESPONSIBLE USE

CodeGen-3D is built on publicly released research datasets (Objaverse), which handle data licensing and content filtering at the dataset level. We use these assets only as evaluation targets, rendering a subset of objects and reporting aggregate metrics without redistributing the underlying 3D models or processing personal data. Our use of LLMs/VLMs is restricted to code generation and automated evaluation for research purposes.

REFERENCES

[1] C. Sun, J. Han, W. Deng, X. Wang, Z. Qin, and S. Gould, "3d-gpt: Procedural 3d modeling with large language models," *arXiv preprint arXiv:2310.12945*, 2023.

[2] L. Makatura, M. Foshey, B. Wang, F. Hähnlein, P. Ma, B. Deng, M. Tjandrasuwita, A. Spielberg, C. E. Owens, P. Y. Chen, A. Zhao, A. Zhu, W. J. Norton, E. Gu, J. Jacob, Y. Li, A. Schulz, and W. Matusik, "Large Language Models for Design and Manufacturing," *An MIT Exploration of Generative AI*, mar 27 2024, https://mit-genai.pubpub.org/pub/nmypmnhs.

[3] P. Müller, P. Wonka, S. Haegler, A. Ulmer, and L. Van Gool, "Procedural modeling of buildings," in *ACM SIGGRAPH 2006 Papers*, 2006, pp. 614–623.

[4] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022.

[5] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg *et al.*, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint arXiv:2303.12712*, 2023.

[6] X. Ma, Y. Bhalgat, B. Smart, S. Chen, X. Li, J. Ding, J. Gu, D. Z. Chen, S. Peng, J.-W. Bian *et al.*, "When llms step into the 3d world: A survey and meta-analysis of 3d tasks via multi-modal large language models," *arXiv preprint arXiv:2405.10255*, 2024.

[7] Y. Du, S. Chen, W. Zan, P. Li, M. Wang, D. Song, B. Li, Y. Hu, and B. Wang, "Blenderllm: Training large language models for computer-aided design with self-improvement," *arXiv preprint arXiv:2412.14203*, 2024.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.

[9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.

[10] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[11] Y. He, Y. Bai, M. Lin, W. Zhao, Y. Hu, J. Sheng, R. Yi, J. Li, and Y.-J. Liu, "T^3 bench: Benchmarking current progress in text-to-3d generation," *arXiv preprint arXiv:2310.02977*, 2023.

[12] Y. Zhang, M. Zhang, T. Wu, T. Wang, G. Wetzstein, D. Lin, and Z. Liu, "3dgen-bench: Comprehensive benchmark suite for 3d generative models," *arXiv preprint arXiv:2503.21745*, 2025.

[13] M. Deitke, D. Schwenk, J. Salvador, L. Weihs, O. Michel, E. VanderBilt, L. Schmidt, K. Ehsani, A. Kembhavi, and A. Farhadi, "Objaverse: A universe of annotated 3d objects," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 13 142–13 153.

[14] T. Luo, C. Rockwell, H. Lee, and J. Johnson, "Scalable 3d captioning with pretrained models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 75 307–75 337, 2023.

[15] T. Luo, J. Johnson, and H. Lee, "View selection for 3d captioning via diffusion ranking," in *European Conference on Computer Vision*. Springer, 2024, pp. 180–197.

[16] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[17] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.

[18] J. Gao, T. Shen, Z. Wang, W. Chen, K. Yin, D. Li, O. Litany, Z. Gojcic, and S. Fidler, "Get3d: A generative model of high quality 3d textured shapes learned from images," *Advances in neural information processing systems*, vol. 35, pp. 31 841–31 854, 2022.

[19] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "Dreamfusion: Text-to-3d using 2d diffusion," *arXiv preprint arXiv:2209.14988*, 2022.

[20] C.-H. Lin, J. Gao, L. Tang, T. Takikawa, X. Zeng, X. Huang, K. Kreis, S. Fidler, M.-Y. Liu, and T.-Y. Lin, "Magic3d: High-resolution text-to-3d content creation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 300–309.

[21] Z. Wang, C. Lu, Y. Wang, F. Bao, C. Li, H. Su, and J. Zhu, "Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation," *Advances in neural information processing systems*, vol. 36, pp. 8406–8441, 2023.

[22] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen, "Point-e: A system for generating 3d point clouds from complex prompts," *arXiv preprint arXiv:2212.08751*, 2022.

[23] H. Jun and A. Nichol, "Shap-e: Generating conditional 3d implicit functions," *arXiv preprint arXiv:2305.02463*, 2023.

[24] R. Liu, R. Wu, B. Van Hoorick, P. Tokmakov, S. Zakharov, and C. Vondrick, "Zero-1-to-3: Zero-shot one image to 3d object," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 9298–9309.

[25] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.

[26] K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu, and M. Gombolay, "Generating cad code with vision-language models for 3d designs," *arXiv preprint arXiv:2410.05340*, 2024.

[27] A. Daareyni, A. Martikkala, H. Mokhtarian, and I. F. Ituarte, "Generative ai meets cad: enhancing engineering design to manufacturing processes with large language models," *The International Journal of Advanced Manufacturing Technology*, pp. 1–10, 2025.

[28] V. Kumaran, J. Rowe, B. Mott, and J. Lester, "Scenecraft: Automating interactive narrative scene generation in digital games with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 19, no. 1, 2023, pp. 86–96.

[29] R. Wu, C. Xiao, and C. Zheng, "Deepcad: A deep generative network for computer-aided design models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6772–6782.

[30] M. S. Khan, S. Sinha, T. Uddin, D. Stricker, S. A. Ali, and M. Z. Afzal, "Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts," *Advances in Neural Information Processing Systems*, vol. 37, pp. 7552–7579, 2024.

[31] R. Wang, Y. Yuan, S. Sun, and J. Bian, "Text-to-cad generation through infusing visual feedback in large language models," *arXiv preprint arXiv:2501.19054*, 2025.

[32] J. Li, W. Ma, X. Li, Y. Lou, G. Zhou, and X. Zhou, "Cad-llama: leveraging large language models for computer-aided design parametric 3d model generation," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 18 563–18 573.

[33] A. M. Anis, H. Ali, and S. Sarfraz, "On the limitations of vision-language models in understanding image transforms," *arXiv preprint arXiv:2503.09837*, 2025.

[34] S. Maiti, L. Agapito, and F. Kokkinos, "Gen3deval: Using vllms for automatic evaluation of generated 3d objects," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 18 552–18 562.

[35] Y. Wu, F. Liu, R. Yilmaz, H. Konermann, P. Walter, and J. Stegmaier, "A pragmatic note on evaluating generative models with fr\'echet inception distance for retinal image synthesis," *arXiv preprint arXiv:2502.17160*, 2025.

[36] F. Lin, Y. Yue, Z. Zhang, S. Hou, K. Yamada, V. Kolachalama, and V. Saligrama, "Infocd: A contrastive chamfer distance loss for point cloud completion," *Advances in Neural Information Processing Systems*, vol. 36, pp. 76 960–76 973, 2023.

[37] X. Liu, C.-K. Tang, and Y.-W. Tai, "Worldcraft: Photo-realistic 3d world creation and customization via llm agents," *arXiv preprint arXiv:2502.15601*, 2025.

[38] Y. Lu, D. Jiang, W. Chen, W. Y. Wang, Y. Choi, and B. Y. Lin, "Wildvision: Evaluating vision-language models in the wild with human preferences," *Advances in Neural Information Processing Systems*, vol. 37, pp. 48 224–48 255, 2024.

[39] Y. Peng, Y. Cui, H. Tang, Z. Qi, R. Dong, J. Bai, C. Han, Z. Ge, X. Zhang, and S.-T. Xia, "Dreambench++: A human-aligned benchmark for personalized image generation," *arXiv preprint arXiv:2406.16855*, 2024.

TABLE 1. Paired comparisons vs. Ground Truth (p–thresholds) and Pearson correlation with Ground Truth. Stars mark the significance of r: $^{*}p<.05$, $^{**}p<.01$, $^{***}p<.001$. All paired tests indicate model scores are lower than Ground Truth.

Model	Average CLIP		Max CLIP	
	t / Wilcoxon (p)	r	t / Wilcoxon (p)	r
GPT-4.1-mini	< 0.001 / < 0.001	0.438^{***}	< 0.001 / < 0.001	0.289^{*}
GPT-4.1	< 0.001 / < 0.001	0.402^{***}	< 0.001 / < 0.001	0.296^{*}
O4-mini	< 0.001 / < 0.001	0.374^{***}	< 0.001 / < 0.001	0.180
BlenderLLM	< 0.001 / < 0.001	0.199	< 0.001 / < 0.001	0.091
Claude Sonnet	< 0.001 / < 0.001	0.547^{***}	< 0.001 / < 0.001	0.403^{*}
Claude Opus	< 0.001 / < 0.001	0.350^{*}	< 0.001 / < 0.001	0.260
Gemini 2.5 Flash	< 0.001 / < 0.001	0.283^{*}	< 0.001 / < 0.001	0.203
Gemini 2.5 Pro	< 0.001 / < 0.001	0.261	< 0.001 / < 0.001	0.182
GPT 5	< 0.01 / < 0.01	0.259	< 0.001 / < 0.01	0.353
Iterative Agent	< 0.001 / < 0.001	0.377^{***}	< 0.001 / < 0.001	0.212^{*}

APPENDIX A ADDITIONAL STATISTICAL ANALYSIS

A. CLIP STATISTICAL SIGNIFICANCE

We tested per–prompt differences between the Ground Truth and each model using paired t–tests and Wilcoxon signed–rank tests on both *Average* and *Max* CLIP scores. As shown in Table 1, all models score *significantly below* Ground Truth. For nearly every comparison, both tests yield $p < 0.001$; the only exception is **GPT 5**, for which the Wilcoxon test is $p < 0.01$ due to the small sample size ($n = 8$).

Pearson correlations (r) between each model's scores and the Ground Truth are generally weak–to–moderate and often significant for *Average* scores (notably **GPT-4.1-mini**, **GPT-4.1**, **O4-mini**, **Claude Sonnet**, and our **Iterative Agent**). In contrast, significance is sparser for *Max* scores (mainly **GPT-4.1-mini**, **GPT-4.1**, **Claude Sonnet**, and **Iterative Agent**), suggesting that maxima are more volatile across prompts.

APPENDIX B RUNTIME AND COMPUTATIONAL COST

In addition to accuracy metrics, we report the computational cost of the CodeGen-3D pipeline. All measurements in this section correspond to the *Blender stage* only, i.e., executing the generated Python script and producing six orthographic renderings. This stage is run on a laptop equipped with an NVIDIA GeForce RTX 4070 Laptop GPU, an Intel Core i9 CPU, and 16 GB of RAM, using Blender 4.4. For each model, we measure the wall-clock time required for this Blender

TABLE 2. Computational cost on CodeGen-3D. For each model, we report the average and median Blender execution + six-view rendering time per successfully generated object (Blender stage only), and the approximate API cost per 100 prompts. Gemini models were run using free public credits, so no USD cost is reported.

Model	Avg. Blender time (s/object)	Median Blender time (s/object)	API cost/100 prompts (USD)
BlenderLLM	16.61	8.74	– (local)
GPT-5	1.28	0.53	$6.50
O4-mini	6.78	7.94	$1.06
GPT-4.1	10.22	12.80	$0.83
GPT-4.1-mini	18.88	16.92	$0.18
Gemini 2.5 Flash	4.85	5.08	N/A
Gemini 2.5 Pro	5.49	7.10	N/A
Claude Sonnet	3.65	0.67	$2.10
Claude Opus	5.15	0.78	$13.48
Iterative Agent	22.97	17.28	$3.16

stage and compute both the mean and median runtime per successfully generated object. These timings explicitly exclude LLM generation latency; for the LLM side, we instead report approximate API cost per 100 prompts. The overall report can be seen in Table .

Across all methods, executing a generated script and rendering six views requires on average 9.6 seconds per object (mean over models), with an average median runtime of 7.8 seconds. Per-model mean runtimes range from 1.28 seconds (GPT-5) to 22.97 seconds (Iterative Agent), while per-model medians range from 0.53 seconds to 17.28 seconds. Although these values vary with the complexity of the generated scene, they remain within a range of a few to tens of seconds on a single RTX 4070 Laptop GPU. Differences in overall deployment cost are primarily driven by the LLM API usage rather than Blender execution: approximate API costs per 100 prompts range from $0.18 for lighter models to $13.48 for the most expensive configuration (Claude Opus), with our iterative agent incurring $3.16 per 100 prompts. For the Gemini models, we do not report a dollar cost because all experiments were conducted using free public API credits.

HAO JI is a Data Scientist at the Center for Advanced Research Computing at University of Southern California. He earned his Master's degree in Mechanical Engineering from the University of California, Berkeley, and his Ph.D. in Mechanical Engineering from USC. Dr. Ji has extensive experience conducting research and applying techniques at the intersection of artificial intelligence (AI) and high-performance computing (HPC). His research interests include fine-tuning large language models and Retrieval-Augmented Generation (RAG) systems, multi-agent reinforcement learning system modeling, deep learning applications in time series analysis and etc. He has authored over ten peer-reviewed publications, covering topics such as deep reinforcement learning, self-organizing systems, and the influence of task characteristics on agent behavior. Dr. Ji serves as a reviewer for several leading journals in engineering and AI, including *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, *Journal of Mechanical Design*, *Computer Aided Design*, and *Automation in Construction*. He is also a regular reviewer for international conferences such as ASME IDETC and PEARC. Dr. Ji received the USC Provost Fellowship and was a torchbearer in the 2008 Beijing Olympic Games. As a participant in multiple NSF-funded projects totaling over 1.6 million. he actively promotes the integration and practical application of artificial intelligence, big data, engineering manufacturing, education, and research computing.

ADITYA KOTHA is an M.S. student in Computer Science (Artificial Intelligence) at the University of Southern California (USC). He received his B.Tech. degree in Computer Science and Engineering from IIIT Naya Raipur in 2024. His research focuses on machine-learning robustness, efficient generative-model inference, and intelligent systems. His broader interests span TinyML for biomedical signals, computer-vision pipelines, and IoT systems. Mr. Aditya has coauthored publications in venues such as *IEEE Transactions on Network and Service Management* and *IEEE Networking Letters*, including work on extreme quantization for P4-enabled IoT networks and Adaptive Continuous Adversarial Training (ACAT) for improving ML robustness.

SEBASTIAN ESCALANTE holds Master of Science in Computer Science from the University of Southern California (USC), specializing in artificial intelligence and applied machine learning. He graduated from the University of Engineering and Technology (UTEC) in Peru with a Bachelor of Science in Electrical Engineering. His research interests include in-depth explorations of large language models, computer vision, and scalable machine learning system architectures. His recent works include evaluation systems for large language models, optimizing evaluation pipelines for accelerating chip verification, and utilizing computer vision for real-time recognition tasks. His broader research interests include large language models and generative AI, computer vision for real-time recognition, and the design of scalable machine-learning systems across modalities such as text, vision, and time-series data. Mr. Escalante's authorships of peer-reviewed articles in IEEE conferences cover topics, ranging from license plate recognition to sensor signal processing. Additionally, he is a participant in professional organizations like IEEE, SHPE, and REPU.

YUNJIAN (JOJO) QIU received the Ph.D. degree in Mechanical Engineering from the University of Southern California. She is currently an Assistant Professor in the Department of Mechanical Engineering at San Jose State University. Her research interests include intelligent and data-driven design, human-centered design, and the integration of AI into mechanical and product design processes. She has worked on projects involving generative AI, machine learning for design exploration and optimization, and autonomous experimentation systems. She has published her work in top journals such as the ASME Journal of Mechanical Design (JMD), ASME Journal of Computing and Information Science in Engineering (JCISE), and ISWA Journal, as well as leading conferences including ASME IMECE and ASME IDETC.